Lab 2 The Sampling Theorem and Fourier Analysis

Objectives:

1. Familiarisation with MATLAB
2. Generation of common sequences.
3. Study of the Sampling Theorem
4. Study of a real-life application of the Fourier Analysis

Report:

You are required to work in a group of no more than two (2) student. A short report covering parts B, C and D detailing your observations and conclusions and including answers to any questions asked in these notes, is to be submitted to the link on the course's Vision site. Try to be as succinct as possible as you can. Use the answer template attached. Pleases clearly write down the details of both of your team members.

Only one (1) report in PDF format is required per group. Please include all your matlab source file(s) for Part D in .m format and not as appendices to your main report.

Start Up:

For this exercise, you will need to download the accompanying MATLAB m-files from the same folder on vision. Please ensure that these files below are stored in a path accessible by MATLAB or in your working folder within the MATLAB IDE.

bv.m: contains the blood velocity data required for illustration

shannon.m: a function that sub-samples an array, x, at regular intervals, T (in samples), and then reconstructs a band-limited signal from the samples using Shannon's formula

$$f(t) = \sum_{n=-\infty}^{\infty} f(nT)\,\mathrm{sinc}\left(\frac{\pi}{T}(t - nT)\right)$$

Your folder should also contain ten (10) .wav file for Part D.

Part A. MATLAB Familiarisation.

MATLAB is a program designed primarily to operate on matrices and vectors. The various ways in which finite sequences can be defined and used in MATLAB will be introduced as needed. This first part of this exercise introduces methods for defining sequences and some basic manipulations on them. It may be (but perhaps should not be) skipped by students who are familiar with MATLAB.

(Note: all command lines below are assumed to be entered by pressing *ENTER.*)

1. The simplest way to construct an array is to list its elements in order.

   Type : *a1 = [1 2 3 4 5]* and observe the result. Type: *a1* to verify that the sequence has been defined. (If you wish to suppress the automatic listing of results, end the line with a semi-colon. Thus : *a1 = [1 2 3 4 5];* defines the array but does not list the elements.)

2. A sequence of equally spaced values can be placed in an array by using the "colon notation". Thus *a2 = [a:b:c]* creates an array (*a2*) with elements starting at *a*, incrementing by *b* and ending at (or before) *c*. If *b* = 1 it may be omitted. The brackets can also be omitted if the result is unambiguous.

   Type: *a2 = [1:1:5]* and observe the result.
   Type: *a3 = [1:5]* and compare the result with *a2*
   Type: *a4 = 0:0.1:2* and observe the results.

3. If a fixed number of equally spaced elements is required the
   linspace(first_value, last_value, Number of values)
   function can be used.

   Type *a5 = linspace(0,20,21)* and observe the result

4. Special arrays can be used for some purposes. For example *zeros(1,n)* generates an n-element array of zeros. Similarly *ones(1,n)* generates a n-element array of ones.

   Type: *a6 = zeros(1,10)* and *a7 = ones(1,10)* and verify the results.

5. Array elements can be addressed by using the equivalent of index addressing. Thus *a1(2)* is the second element in the array a1, while *a3([1 2 4])* stands for a 3 element array consisting of the first, second and fourth elements of the array a3 and *a4([1:2:15])* stands for the elements of the array a4 with indices 1,3,5,7,9,11,13 and 15.

   Type: *a8 = a4([1:2:15])* and note the result
   Type: *a8(4) = 0* and note the result.

6. An array may be multiplied or divided by a scalar: Try *a = a2*3* and *a = a2/3* and note the results.

7. A scalar may be added to or subtracted from an array. Try *a = a2 + 1* and *a = a2 - 4*.

8. Arrays may be added or subtracted : Try *a = a2 + a3* and *a = a2 - a3*

9. Arrays may be multiplied or divided (term by term) by using the .* or ./ operators. Try *a = a2.*a3* and *a = a2. / a3*. (Make sure to include the '.')

10. Logical relations can also be applied (term by term) to produce arrays of 0 (= FALSE) and 1 (= TRUE) values. The logical operators available are

    < less than
    > greater than
    <= less than or equal
    >= greater than or equal
    == equal
    ~= not equal.

    Try *a = (a1 > 3)* and *a = (a1 == 3)* and *a = (a1 < 3)* and *a = (a1 == a2)*.

11. Logical operators , & (AND) , | (OR) and ~ (NOT) can be applied to (0,1) arrays.

    Try *a = (a1 > 3)&(a1 < 3)* and *a = (a1 > 3)|(a1 < 3)* and *b = ~a*

12. Pairs of arrays can be plotted against one another using the various plot commands. The *plot(x,y)* command plots the contents of the y array against the contents of x array by interpolating straight line segments between the array points.

    Try *x = [1:1:50], y = x.*x, plot(x,y)*

    For discrete arrays it is often more convenient to plot the individual values. The *stem(x,y)* command does this.

    Try *stem(x,y)*.

    A histogram-like plot may also be useful; try *stairs(x,y)*.

    Matlab has other visualisation tool that might be useful in other parts of this course or other courses. You can explore the usage of histogram (hist(y)) and bar(x,y).

    Compare your plots produced using stairs(x,y) and bar(x,y). What are the differences?

    Optional Activities
    Adjust the bin size parameter for hist(y,binsize). How does this parameter affect the histogram produced?

Part B. Generation of Common Sequences

A number of sequences are commonly used in Discrete Signal Processing applications. As an introductory MATLAB exercise generate and plot the following sequences, each consisting of 50 samples, the sample numbers running over the integers $-10 \le n \le 39$.

1. Unit sample or Unit Impulse sequence.

(a). $x1a = \delta(n) = \begin{cases} 1 \ ; & n = 0 \\ 0 \ ; & n \neq 0 \end{cases}$

(b). $x1b = \delta(n-3) = \begin{cases} 1 \ ; & n = 3 \\ 0 \ ; & n \neq 3 \end{cases}$

(c). $x1c = \delta(n+4) = \begin{cases} 1 \ ; & n = -4 \\ 0 \ ; & n \neq -4 \end{cases}$

Hints: First generate a sequence *n = [-10:1:39]* to represent the sample numbers.

THEN
Generate an array (*x1a*) of 50 zeros, and then set the element of the appropriate index to 1
OR
construct the sequence *x1a = (n == m)* for the appropriate m.

Plot the sequence using *stem(n,x1a)*

Question 1: Explain why the second method (i.e. why *x1a=(n==m)*) for constructing the unit sample sequence works. (5%)

2. Unit step sequence.

(a). $x2a = u(n) = \begin{cases} 1 \ ; & n \ge 0 \\ 0 \ ; & n < 0 \end{cases}$

(b). $x2b = u(n-2) = \begin{cases} 1 \ ; & n \ge 2 \\ 0 \ ; & n < 2 \end{cases}$

Construct and plot the two sequences above for $-10 \le n \le 39$.

Hints: An array can be constructed by concatenating two arrays, thus *x2a = [zeros(1,k1) ones(1,k2)]* consists of *k1* zeros followed by *k2* ones. Alternatively, *x2a = (n >= 0)* produces a unit step with a transition at n = 0.

A third (somewhat less elegant) possibility uses the function *b = sign(a)* which returns an array of the same size as *a*, but with *b(k) = -1* if *a(k) < 0*, *b(k) = 0* if *a(k) = 0* and *b(k) = 1* if *a(k) > 0*. Try *x2a = sign(1 + sign(n))*

Question 2: Explain why *x2a = sign(1 + sign(n))* is equivalent to the unit step sequence. (5%)

3. Unit rectangular pulse sequence.

(a). $x3a = P_{10}(n) = \begin{cases} 1 & : \ 0 \le n \le 10 \\ 0 & : \ n < 0 \ \text{ or } \ n > 10 \end{cases}$

(b). $x3b = P_{10}(n+5)$

(c). $x3c = P_{10}(3-n)$

Construct and plot the sequences above for $-10 \le n \le 39$

(Hints: The unit pulse sequence can be related to the unit step by $P_k(n) = u(n) - u(n-(k+1))$ or can be constructed logically by *Pk = ((n >= 0)&(n <= k)).*)

4. Unit height square wave of period 1ms sampled at a rate of 8 kHz.

$x4 = S_8(n) = \begin{cases} 1 & : \ 8k \le n \le 8k+3 \ \text{ for any integer } k \\ 0 & : \ \text{otherwise} \end{cases}$

Construct and plot the sequence above for $-10 \le n \le 39$.

(Hints: construct by hand an array (r) equal to one cycle of the square wave, and then generate *x4* by concatenating the appropriate number of repetitions of r together with the correct parts of a cycle at either end of the array.)

5. Sawtooth wave of period 1ms sampled at 8 kHz.

(a). *x5a = sawtooth wave as described above.*

One cycle of the continuous sawtooth wave is defined by $f(t) = \dfrac{t}{T} \ : \ 0 \le t < T$,

where T is the period and t the time in seconds.

(b). $x5b = \begin{cases} x5a & : \ 0 \le n \le 10 \\ 0 & : \ n < 0 \ \text{ or } \ n > 10 \end{cases}$

Construct and plot the two sequences above for $-10 \le n \le 39$.

(Hints: Determine the sampled values for one cycle and proceed as in (4) above. For (b). consider writing *x5b = x5a.*p* where *p* is an appropriate sequence.)

Question 3: What is the appropriate sequence required for *p* in part (b) ? (5%)

Part C : The Sampling Theorem.

The Sampling Theorem says that a band limited signal can be sampled without loss of information, provided the sampling rate is greater than twice the cutoff frequency of the signal, and that the original signal can be exactly reconstructed (using Shannon's formula or an equivalent method).

If the signal is undersampled ($f_S \leq 2 f_C$) errors in reconstruction result. This error process is called "Aliasing". One way to understand it is to imagine that the high frequency components of the signal fold back at half the sampling frequency and distort the lower frequency components.

Aliasing shows up as a difference between the original signal and the interpolated (reconstructed) signal - not in the sampled values themselves, although in extreme cases it may be evident from the sampled values that aliasing may have occurred.

The aim is to investigate the effects of sampling on the reconstructed signal.

This section uses a function *shannon(x,t,T)* - a MATLAB m-file written for this exercise. This command sub-samples an array, *x*, at regular intervals, *T* (in samples), and then reconstructs a band-limited signal from the samples using (an approximation to) Shannon's formula. It then displays the original array, the reconstructed array and the sampled points on the same plot. The array *t* serves as the time axis on the plot.
(The details of the operation of *shannon* are not, for the moment, important.)

(a). Generate a "time axis" sequence (*t*) with 512 values and a data sequence (*x*) of 512 values containing 8 cycles of a sine wave.

$(t = [0{:}511] * 2 * pi / 512; x = \sin(t * 8); plot(t, x);)$

(b). Calculate the sub-sampling interval corresponding to the "Nyquist rate" for the signal generated in (a) (ie the interval required to ensure two samples per cycle of the signal) and observe the effects of sampling above this rate by setting T to half this value and using the *shannon(x,t,T)* command..

(c). Repeat (b) for sampling at the Nyquist rate and below the Nyquist rate: (set T to the value calculated in (b) and then to 3/2 times this value).

Question 4: Explain your observations in (b) and (c) by considering the problem of sampling a harmonic signal. (15%)

(d). Generate a 4-cycle square wave in *x*, and use *shannon(x,t,T)* to observe the effect of various sampling rates.
(Hint: To construct the square wave : *r = [ones(1,64) zeros(1,64)];*
x = [r r r r];)

Question 5: Explain why the reconstructions in (d) are expected to be more distorted than those at the same sampling rate in (b), even though the fundamental frequency in (d) is less than that in (b). (15%)

(e). Read in the data for a single cycle of blood flow velocity (type the command *bv;* the data will be placed in the *x* array). Observe the effect of sub-sampling and reconstruction at a range of sample intervals using the *shannon* command.

Part D Fourier Analysis

Dual Tone Multi Frequency (DTMF) signals are commonly heard on telephone dial pads. Each key on the dial pad generate a signal that consists of two sinusoids of different frequency. Table 1 below shows the frequency matrix that is used to generate the sound.

Table 1 DTMF Generator Matrix

	1209 Hz	**1336 Hz**	**1477 Hz**
667 Hz	1	2	3
770 Hz	4	5	6
852 Hz	7	8	9
941 Hz	*	0	#

The .wav files *pin_0.wav* to *pin_9.wav* each contain a 4-digit pin code entered and recorded through a DTMF enabled telebanking system.

Select the corresponding wav file based on the follow formulae:

Z = ceil(X+Y/2);

Where Z is the index of the Wav file i.e. Wave{Z}.wav
Y is the last digit of Student ID 1
X is the last digit of Student ID 2

Note: If you are unable to form a group, then use the

Perform Fourier Analysis on the wav file and determine the stored 4-digit pin number.

You should also produce the Magnitude Spectrum of each digit in a combined figure containing 4 subfigures. The title of the individual subfigure denotes the corresponding digit of the Pin.

Other information:

1. You can read in the .wav files using *audioread*.
2. Each of the tone were recorded with constant pulse with and separate by a fix interval (which is not the same as the pulse with). You need to decide how to isolate the individual tone and justify for the parameters you used.
3. In MATLAB, the discrete Fourier Transform (DFT) is computed through the function fft.m. You can refer to its documentation through help *fft*.
4. To enhance the magnitude spectra produced directly through DFT/FFT, windowing the region of analysis will help[1] to reduce spectral leakage.
   Choose an appropriate window function and justify for your choice.
   Discuss the effect of different window width (i.e. the number of samples) used for DFT computation on the resulting magnitude spectrum.
   (55%)

[1] https://www.edn.com/windowing-functions-improve-fft-results-part-i/

Part D Marks Breakdown

20%	for correct determination of technical parameters (Sampling Frequency, Pulse width, Pulse spacing) and correct pin decoded.
10%	for the accuracies of plots produced.
5%	MATLAB Program and its Documentation
20%	for Justification of choice of window function and discussion on spectral leakage

END OF LAB 2